SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

SET 2:

1.A retail company collects terabytes of customer transaction data every day from online and offline stores. The management wants to analyze purchasing patterns in near real time to improve product recommendations and inventory management.

- Design a cloud-based big data architecture using distributed storage and processing frameworks.
- Explain how batch processing and stream processing can be integrated.
- Discuss how data lakes and data warehouses can be used together.
- Recommend techniques to ensure scalability and cost efficiency.

2.A healthcare organization is migrating patient records and medical imaging data to the cloud. The solution must comply with strict privacy regulations while allowing authorized doctors to access records securely from multiple locations.

- Design a secure cloud architecture that protects sensitive healthcare data.
- Explain how encryption, identity and access management (IAM), and key management should be implemented.
- Discuss auditing, logging, and compliance monitoring strategies.
- Recommend methods to prevent unauthorized access and data breaches.

3.An online ticket booking platform experiences sudden spikes in traffic whenever popular events are announced. The company wants to reduce infrastructure management while ensuring fast response times.

- Design a serverless cloud architecture for the application.
- Explain how event-driven computing can improve scalability.
- Discuss the role of managed databases, API gateways, and message queues.
- Evaluate the advantages and limitations of serverless computing for this workload.

4.A multinational company gathers data from sales, finance, manufacturing, and customer support systems. The organization requires a centralized analytics platform with high-quality and consistent data.

- Design a cloud-based ETL/ELT pipeline for integrating data from multiple sources.
- Explain how data quality, metadata management, and governance should be maintained.
- Discuss the role of workflow orchestration and scheduling tools.
- Recommend strategies for optimizing query performance and storage costs.

5.A logistics company wants to predict delivery delays using historical shipment data, weather information, and live GPS tracking. The analytics platform must support continuous model training and real-time predictions.

- Design a cloud-based machine learning and analytics architecture.
- Explain how big data technologies can support model training and inference.
- Discuss the use of cloud-native AI/ML services for model deployment and monitoring.
- Recommend strategies for handling large-scale data ingestion, feature engineering, and model updates.

SET 3 :

1. A manufacturing company plans to migrate its legacy enterprise applications and databases from an on-premises data center to the cloud. The organization wants to minimize downtime while ensuring application performance and security.

- Design a cloud migration strategy suitable for the organization.
- Explain how different migration approaches (Rehost, Refactor, Replatform, Repurchase, Retire, Retain) can be applied.
- Discuss methods to validate application performance after migration.
- Recommend strategies to reduce migration risks and downtime.

2. A smart city project deploys thousands of IoT sensors to monitor traffic, air quality, energy consumption, and public safety. The collected data must be processed and analyzed in real time for decision-making.

- Design a cloud-based architecture for IoT data collection and analytics.
- Explain how message brokers, stream processing, and cloud storage are integrated.
- Discuss methods for handling high-volume, high-velocity sensor data.
- Recommend techniques to ensure scalability, reliability, and security.

3. A global e-commerce company wants executives to monitor sales performance, customer behavior, and regional trends through interactive dashboards. The solution must support real-time analytics and historical reporting.

- Design a cloud-based business intelligence architecture.
- Explain how data visualization tools integrate with cloud data warehouses.
- Discuss techniques for optimizing dashboard performance with large datasets.
- Recommend security measures to protect business reports and sensitive data.

4. An online gaming company experiences fluctuating workloads throughout the day. During peak hours, users report high latency and reduced application performance.

- Design a cloud monitoring and performance optimization solution.

- Explain how cloud monitoring tools, metrics, logs, and tracing help identify performance bottlenecks.
- Discuss strategies for resource optimization and capacity planning.
- Recommend methods to improve application availability while controlling operational costs.

5. A financial institution stores petabytes of customer transaction data for regulatory compliance and business analytics. The organization must retain historical records while managing storage costs efficiently.

- Design a cloud-based data lifecycle management strategy.
- Explain how data classification, retention policies, and archival storage should be implemented.
- Discuss the importance of governance, auditing, and regulatory compliance in big data environments.
- Recommend techniques for balancing data accessibility, security, and cost optimization.

Question 1: Video Streaming Platform Migration

(a) Cloud Architecture Using Scalability and Caching

The media company must serve millions of concurrent viewers with minimal buffering. The proposed architecture is a multi-tier, horizontally scalable design with a CDN at the edge.

- **Ingest and transcoding:** Raw video is uploaded to object storage (e.g., Amazon S3 / Azure Blob). A queue-driven transcoding fleet converts each title into multiple bitrates and formats (HLS/DASH) for adaptive bitrate streaming.
- **Storage:** Encoded segments are stored in durable object storage with lifecycle tiers (hot, infrequent, archive). Metadata (catalogue, user profiles, watch history) lives in a scalable NoSQL database such as DynamoDB or Cassandra.
- **Edge caching:** A Content Delivery Network (CloudFront / Akamai / Azure CDN) caches video segments at edge locations close to viewers, so most requests never reach the origin. An origin shield layer reduces origin load further.
- **Application tier:** Stateless microservices (login, search, recommendations, playback authorisation) run in containers or VMs across multiple Availability Zones behind load balancers.
- **In-memory caching:** Redis/Memcached caches session data, popular catalogue pages and recommendation results to cut database reads and reduce latency.
- **Observability and security:** Monitoring, logging, DRM, signed URLs and WAF protect content and detect problems early.

(b) Load Balancing and Auto-Scaling for Smooth Streaming

Load balancing: Global DNS routing (e.g., Route 53 latency-based) sends each user to the nearest healthy region. Within a region, an Application Load Balancer spreads requests across instances in several Availability Zones, performs health checks, and removes failed nodes automatically. This avoids hot spots and single points of failure.

Auto-scaling: Auto-scaling groups add or remove instances based on metrics such as CPU, request count per target and concurrent streams. Predictive or scheduled scaling prepares capacity before known peaks (a new season release, a live sports event), while target-tracking policies handle unexpected surges. Scale-in rules and cooldown periods prevent flapping.

Result: Combined with adaptive bitrate streaming, which lowers quality instead of stalling when bandwidth drops, viewers get consistent playback with minimal buffering even at millions of concurrent users.

(c) Cost Optimisation Strategies

Strategy	How it saves cost
Maximise CDN cache hit ratio	Fewer origin requests and lower egress fees; negotiate committed-volume CDN pricing.
Reserved / Savings Plans	Cover the predictable baseline load at up to 60-70% discount.
Spot / preemptible instances	Use for fault-tolerant transcoding jobs at large discounts.
Storage tiering	Move old or rarely watched titles to infrequent-access or archive tiers automatically.
Efficient encoding	Use modern codecs (H.265/AV1) and per-title encoding to reduce bandwidth.
Right-sizing and scheduling	Review utilisation regularly; shut down idle non-production resources.
Cost monitoring	Tagging, budgets and alerts (Cost Explorer) to detect anomalies early.

Question 2: Multi-Cloud Container Deployment for a Fintech Startup

(a) Solution Using Container Orchestration and Multi-Cloud

The startup should standardise on Kubernetes as the common orchestration layer across all providers, so the same workloads run identically on AWS (EKS), Azure (AKS) and Google Cloud (GKE).

- **Containerisation:** Every service is packaged as an immutable Docker image stored in a central, replicated registry. This removes 'works on my machine' inconsistency.
- **Infrastructure as Code:** Terraform provisions clusters and networks identically in each cloud; Helm charts or Kustomize define application releases.
- **GitOps and CI/CD:** Argo CD or Flux keeps cluster state in sync with a Git repository, giving consistent, auditable, repeatable deployments and easy rollback.
- **Central management:** A multi-cluster management tool (Rancher, Anthos, Azure Arc) gives one control plane for policy, RBAC and visibility.
- **Security and compliance:** Policy-as-code (OPA Gatekeeper), image scanning, secrets management (Vault) and network policies, essential for fintech regulation.

(b) Service Discovery and Scaling

Service discovery: Kubernetes assigns each Service a stable virtual IP and DNS name (CoreDNS), so pods find each other even as they are rescheduled. A service mesh (Istio / Linkerd) adds mTLS, traffic routing, retries and observability. For cross-cluster and cross-cloud discovery, mesh federation or tools such as Consul and Cilium Cluster Mesh are used, with a global load balancer / DNS directing external traffic.

Scaling: The Horizontal Pod Autoscaler adds or removes pods based on CPU, memory or custom metrics (e.g., transactions per second). The Cluster Autoscaler (or Karpenter) adds nodes when pods cannot be scheduled. KEDA can scale on queue length. Resource requests and limits keep workloads efficient and prevent noisy neighbours.

(c) Benefits and Risks of Multi-Cloud

Benefits	Risks
Avoids vendor lock-in and improves bargaining power	Higher operational complexity and need for broader skills
Higher resilience: survives a provider-wide outage	Inconsistent security models and IAM across providers
Best-of-breed services and regional/data-residency coverage	Data egress charges and cross-cloud latency
Compliance flexibility for regulated fintech workloads	Harder cost tracking and governance; larger attack surface
Cost optimisation by choosing cheaper options	Lowest-common-denominator design may waste provider-specific features

Question 3: Backup and Disaster Recovery for an E-Learning Platform

(a) Solution: Cloud-Native Backup, Replication and Versioning

- **Versioning and delete protection:** Enable object versioning on storage buckets so accidental deletions create a delete marker rather than removing data. Add MFA-Delete and object lock (WORM) for critical content, and restrict delete permissions through least-privilege IAM.
- **Automated backups:** Use AWS Backup / Azure Backup for databases, volumes and file systems with scheduled, policy-based backups, snapshots and point-in-time recovery (PITR) for the course and student databases.
- **Replication:** Cross-AZ replication for high availability, plus cross-region replication of backups, storage and database read replicas to survive a regional failure. Store a copy in a separate account for protection against ransomware and credential compromise.
- **Retention and lifecycle:** Daily, weekly and monthly retention rules, with older backups moved to archive tiers (Glacier) for cost savings.
- **Testing and monitoring:** Scheduled restore tests, backup-job alerts and audit logging (CloudTrail).

(b) Achieving RTO and RPO

RPO (Recovery Point Objective): the maximum acceptable data loss, measured in time. It is met by backup and replication frequency. Continuous replication and PITR give an RPO of seconds to minutes; hourly snapshots give about one hour. Versioning gives near-zero RPO for deleted objects.

RTO (Recovery Time Objective): the maximum acceptable downtime. It is met by the chosen DR pattern:

DR pattern	Typical RTO	Typical RPO	Cost
Backup and restore	Hours	Hours	Lowest
Pilot light	Tens of minutes to hours	Minutes	Low
Warm standby	Minutes	Seconds to minutes	Medium
Multi-site active-active	Near zero	Near zero	Highest

For the e-learning platform, warm standby for the learning application and database, with versioned storage for course material, balances cost and recovery goals (for example RTO of 15 to 30 minutes and RPO of under 5 minutes). Infrastructure as Code and automation make the recovery repeatable and fast.

(c) Steps to Ensure Business Continuity During Failures

1. Maintain a documented DR plan with defined roles, contact lists and escalation paths.
2. Detect the failure through health checks, monitoring and alerts.
3. Declare the incident and activate the DR plan.
4. Fail over traffic using DNS (Route 53 health checks) or a global load balancer to the standby region.
5. Restore data from the latest replica, snapshot or object version; promote the standby database to primary.
6. Validate application functionality, data integrity and user logins before announcing recovery.
7. Communicate status to students, teachers and stakeholders (status page, email).
8. Fail back to the primary region once it is stable and data is resynchronised.
9. Hold a post-incident review, update the plan, and run regular DR drills and restore tests.

Question 4: Hybrid Cloud for a Government Agency

(a) Hybrid Architecture with Secure Communication

- **Workload placement:** Sensitive data and compliance-bound applications remain in the on-premises data center (private cloud); non-sensitive, elastic workloads such as public portals, analytics on anonymised data and backups run in the public cloud.
- **Secure connectivity:** A dedicated private link (AWS Direct Connect / Azure ExpressRoute) plus IPsec site-to-site VPN as backup. Traffic never needs to cross the public internet.
- **Network design:** Hub-and-spoke or landing-zone topology, network segmentation with VPCs/VNets, subnets, security groups, firewalls, and private endpoints. A DMZ and API gateway control exposed services.
- **Zero-trust approach:** Every request is authenticated and authorised regardless of location; micro-segmentation limits lateral movement.
- **Consistent platform:** Kubernetes or hybrid platforms (Azure Arc, AWS Outposts, Anthos) provide a uniform operating model across both environments.
- **Monitoring:** A central SIEM collects logs from both environments for threat detection.

(b) Identity Management and Encryption

Identity management: Federate on-premises Active Directory with the cloud identity provider (Azure AD / AWS IAM Identity Center) using SAML/OIDC for single sign-on. Enforce multi-factor authentication, role-based access control with least privilege, privileged access management and just-in-time access, and regular access reviews. Use service identities and short-lived credentials instead of static keys.

Encryption: Data in transit: TLS 1.2+ and IPsec/MACsec on the links. Data at rest: AES-256 on storage, databases and backups. Keys are managed in a KMS, backed by an on-premises Hardware Security Module (HSM) so the agency holds key custody (BYOK / HYOK); keys are rotated regularly and separated by duty. Sensitive fields can additionally use tokenisation or masking.

(c) Challenges in Managing Hybrid Cloud

- **Integration complexity:** Different tools, APIs and architectures between legacy on-premises systems and cloud services.
- **Security and compliance:** Consistent policy enforcement, a larger attack surface and proving compliance across both environments.
- **Visibility and monitoring:** Fragmented logs and metrics make end-to-end troubleshooting difficult.
- **Latency and data movement:** Performance impact and transfer costs between sites; data gravity.
- **Skills and operations:** Staff need expertise in both environments; processes must be unified.
- **Cost management and governance:** Tracking spend and avoiding shadow IT.
- **Disaster recovery:** Coordinating backup and failover across two platforms.

Question 5: High Availability and Fault Tolerance for a SaaS Provider

(a) Multi-Region Deployment Strategy

To keep running if an entire region fails, the application is deployed in at least two geographically separate regions, each spanning multiple Availability Zones.

- **Active-active (preferred):** Both regions serve live traffic; a global load balancer or latency/health-based DNS (Route 53, Azure Front Door, Global Accelerator) routes users to the nearest healthy region.
- **Active-passive (lower cost alternative):** The secondary region runs a scaled-down warm standby that scales up on failover.
- **Stateless application tier:** Containers or auto-scaled instances with externalised sessions (Redis) so any region can serve any user.
- **Redundancy inside each region:** Multi-AZ load balancers, databases and storage eliminate single points of failure.
- **Automation:** Identical stacks built with Infrastructure as Code and deployed through CI/CD (blue-green or canary releases) to avoid configuration drift.

(b) Data Replication and Failover Configuration

Data replication: Use globally replicated databases (Aurora Global Database, DynamoDB Global Tables, Cosmos DB, Spanner). Asynchronous replication gives low latency with a small RPO (seconds); synchronous replication gives RPO of zero but higher write latency. Object storage uses cross-region replication, and conflict resolution rules (last-writer-wins or application logic) are defined for active-active writes. Replication lag is monitored continuously.

Failover: Health checks on endpoints detect regional failure; DNS with low TTL (30-60 s) or anycast routing automatically shifts traffic. In active-passive mode, the standby database replica is promoted to primary automatically or by runbook, and auto-scaling expands the standby fleet. Failover is protected with timeouts, circuit breakers and retries, and the whole process is tested regularly through game days and chaos engineering (e.g., Chaos Monkey, AWS Fault Injection Simulator). After recovery, controlled failback resynchronises data.

(c) Monitoring and Alerting for Proactive Detection

- **Metrics:** Track latency, error rate, throughput, saturation (the four golden signals), plus replication lag and database health, with tools like CloudWatch, Prometheus and Grafana.
- **Centralised logging and tracing:** ELK/OpenSearch, Splunk or Datadog with distributed tracing (OpenTelemetry, X-Ray) to locate bottlenecks across services and regions.
- **Synthetic monitoring:** Scripted probes from multiple locations simulate user journeys and catch outages before customers do.
- **Alerting:** Threshold and anomaly-based alerts tied to SLOs, with severity levels, escalation policies and on-call tools (PagerDuty, Opsgenie), delivered by SMS, email or chat.
- **Dashboards and runbooks:** Real-time dashboards and documented runbooks reduce mean time to detect and repair.
- **Review:** Post-incident reviews and capacity trends feed continuous improvement.

SET 2 ANSWERS

Set 2 - Question 1: Retail Big Data Analytics

(a) Cloud-Based Big Data Architecture

Transaction data from online and offline stores is ingested, stored and processed on distributed, elastic services.

- **Ingestion:** Apache Kafka / Kinesis / Event Hubs for streaming events from POS systems and websites; batch loaders (Sqoop, Glue, Data Factory) for store databases.
- **Storage:** Distributed object storage (S3 / ADLS / GCS) or HDFS as the data lake, in columnar formats such as Parquet.
- **Processing:** Apache Spark (EMR / Databricks / Dataproc) for batch, and Spark Structured Streaming or Flink for real-time.
- **Serving:** A cloud data warehouse (Redshift / BigQuery / Synapse / Snowflake) for BI, and a low-latency store (Redis / DynamoDB) for recommendations.

(b) Integrating Batch and Stream Processing

The Lambda architecture runs a batch layer (accurate, complete, recomputed periodically) and a speed layer (low-latency, approximate) and merges both in a serving layer. The simpler Kappa architecture treats everything as a stream and replays the log for reprocessing. Modern platforms use Spark or Flink with Delta Lake / Iceberg, so one engine and one table format handle both modes. Streaming results update recommendations and stock levels within seconds, while nightly batch jobs correct and enrich the history.

(c) Data Lake and Data Warehouse Together

The lake stores all raw and semi-structured data cheaply (schema-on-read) for data science and ML. Cleaned, modelled and aggregated data is loaded into the warehouse (schema-on-write) for fast SQL and dashboards. Combining them in a lakehouse pattern (raw, curated and presentation zones) gives flexibility, governance and performance, and avoids duplicate copies.

(d) Scalability and Cost Efficiency

- **Scalability:** Partition Kafka topics and Spark data, use auto-scaling clusters and decoupled storage and compute.
- **Cost:** Spot instances for batch, transient clusters shut down after jobs, storage tiering, columnar compression, partition pruning, and reserved capacity for steady workloads.

Set 2 - Question 2: Secure Healthcare Cloud

(a) Secure Cloud Architecture

A defence-in-depth design placed in a compliant region (HIPAA / GDPR / local health regulations).

- **Network isolation:** Private subnets, no public database access, private endpoints, WAF and firewalls, and VPN or private link for hospital sites.

- **Storage:** Encrypted object storage for imaging (DICOM) and encrypted managed databases for patient records.
- **Segmentation:** Separate accounts or subscriptions for production, logging and security.

(b) Encryption, IAM and Key Management

- **Encryption:** AES-256 at rest and TLS 1.2+ in transit, with field-level encryption for identifiers.
- **IAM:** Federated SSO, mandatory MFA, role-based and attribute-based access (doctors see only their patients), least privilege, and break-glass emergency access with review.
- **Key management:** A cloud KMS or HSM with customer-managed keys, automatic rotation, separation of duties, and key access logging.

(c) Auditing, Logging and Compliance Monitoring

- **Audit logs:** Record every access to patient data (who, what, when, from where) using CloudTrail / Azure Monitor, stored immutably.
- **SIEM and alerting:** Centralised analysis for abnormal access patterns.
- **Compliance tools:** Config rules, Security Hub or Defender for continuous checks, plus periodic audits and reports.

(d) Preventing Unauthorised Access and Breaches

- **Prevention:** Zero trust, MFA, DLP, data masking, regular patching and vulnerability scanning.
- **Detection and response:** Intrusion detection, anomaly detection, an incident response plan and breach notification procedure.
- **People and process:** Staff training, access reviews, immutable backups against ransomware and vendor risk assessment.

Set 2 - Question 3: Serverless Ticket Booking

(a) Serverless Architecture

- **Frontend:** Static site on object storage with a CDN.
- **API layer:** API Gateway handles authentication, throttling and routing to functions.
- **Compute:** Functions (Lambda / Azure Functions) for search, seat selection, booking and payment.
- **Data:** DynamoDB or Aurora Serverless for bookings; Redis for seat holds.
- **Messaging:** SQS / SNS / EventBridge for order processing, e-mail and ticket generation.

(b) Event-Driven Computing and Scalability

Each action (booking request, payment confirmed, queue message) triggers a function, so capacity scales automatically from zero to thousands of parallel executions during a sale and back down afterwards. Components are loosely coupled, so a slow email service cannot slow bookings, and users pay only for execution time.

(c) Role of Managed Databases, API Gateway and Queues

- **Managed databases:** Auto-scaling, replication and backups without administration; conditional writes prevent double booking.
- **API gateway:** Rate limiting, caching, authentication, WAF integration and versioning.
- **Message queues:** Buffer traffic spikes, smooth load on payment systems, enable retries and dead-letter queues, and create a virtual waiting room.

(d) Advantages and Limitations

- **Advantages:** Automatic scaling, no server management, pay-per-use, high availability and fast development.
- **Limitations:** Cold-start latency, execution time limits, vendor lock-in, harder debugging and testing, concurrency limits and unpredictable cost at constant high load. Mitigation: provisioned concurrency, caching and reserved concurrency limits.

Set 2 - Question 4: ETL/ELT Analytics Platform

(a) ETL/ELT Pipeline Design

- **Sources:** ERP, finance, MES and CRM/support systems via connectors, CDC (Debezium / DMS) and APIs.
- **Landing and raw zone:** Data lands unchanged in the data lake.
- **Transform:** ELT with dbt / Spark inside the warehouse produces cleaned, standardised, business-ready models (bronze, silver, gold layers).
- **Consumption:** Central warehouse for BI, ML and reporting.

(b) Data Quality, Metadata and Governance

- **Data quality:** Validation rules, deduplication, standard formats and tests (Great Expectations, dbt tests) with quarantine of bad records.
- **Metadata:** A data catalogue (Glue Catalog, Purview, Collibra) with lineage, definitions and ownership.
- **Governance:** Data stewards, master data management, access policies, classification, privacy compliance and audit trails.

(c) Workflow Orchestration and Scheduling

Tools such as Apache Airflow, AWS Step Functions, Azure Data Factory and Prefect define pipelines as dependency graphs (DAGs), schedule or trigger runs, retry failed tasks, manage backfills, send alerts and give monitoring dashboards, which keeps pipelines reliable and repeatable.

(d) Query Performance and Storage Cost

- **Performance:** Partitioning, clustering, columnar formats, materialised views, result caching, and pre-aggregated tables.
- **Cost:** Compression, lifecycle tiering, removing unused data, auto-suspend warehouses and workload isolation.

Set 2 - Question 5: ML Platform for Delivery Delays

(a) ML and Analytics Architecture

- **Ingestion:** GPS streams via Kafka / Kinesis / IoT Core; weather via API batch pulls; shipment history from databases.
- **Storage:** Data lake plus a feature store (SageMaker Feature Store / Feast) with offline and online stores.
- **Training:** Spark and managed ML services (SageMaker, Vertex AI, Azure ML).
- **Serving:** Real-time endpoints behind an API with stream scoring.

(b) Big Data for Training and Inference

Spark and distributed GPU or CPU clusters train on large historical data in parallel, while stream processors (Flink / Spark Streaming) join live GPS and weather data with features and call the model for real-time predictions. Results are stored for dashboards and alerts.

(c) Cloud-Native AI/ML Services

Managed services provide pipelines, experiment tracking, a model registry, one-click deployment, auto-scaling endpoints, A/B or canary releases, and monitoring for data drift, model drift and accuracy, with alerts that trigger retraining.

(d) Ingestion, Feature Engineering and Model Updates

- **Ingestion:** Partitioned streams, schema registry and back-pressure handling.
- **Feature engineering:** Reusable features (traffic, route distance, rolling averages, weather) in a feature store to keep training and serving consistent.
- **Model updates:** MLOps pipelines with scheduled or drift-triggered retraining, validation, CI/CD and rollback.

SET 3 ANSWERS

Set 3 - Question 1: Legacy Application Migration

(a) Migration Strategy

Follow a phased approach: assess and discover (inventory, dependencies, TCO), plan a landing zone, migrate in waves (low-risk first), test, cut over and optimise. Use data replication tools (AWS DMS / Azure Migrate) to keep downtime minimal.

(b) Applying the 6 Rs

- **Rehost:** Lift and shift stable legacy servers to VMs quickly.
- **Replatform:** Move databases to managed services (RDS) with minor changes.
- **Refactor:** Re-architect critical applications into microservices or containers for long-term benefit.
- **Repurchase:** Replace old ERP or CRM with SaaS.
- **Retire:** Decommission unused applications.
- **Retain:** Keep applications with compliance or latency constraints on-premises.

(c) Validating Performance After Migration

Compare against pre-migration baselines using load and stress testing, functional and integration tests, user acceptance testing, monitoring of latency, throughput and error rates, and database consistency checks.

(d) Reducing Risk and Downtime

- **Techniques:** Pilot migrations, continuous data replication, blue-green or canary cutover, rollback plan, backups, staged cutovers in low-traffic windows, security review and staff training.

Set 3 - Question 2: Smart City IoT Analytics

(a) Architecture

- **Device layer:** Sensors with edge gateways for local filtering.
- **Ingestion:** MQTT through IoT Core / IoT Hub with device identity.
- **Processing:** Message broker (Kafka) feeding stream processing (Flink / Spark / Kinesis Analytics).
- **Storage and analytics:** Time-series database and data lake; dashboards and ML.

(b) Integration

Devices publish to a broker; broker topics feed the stream processor, which cleans, aggregates and detects events in real time; hot results go to a time-series store and dashboards, and raw data is archived in cloud storage for batch analytics.

(c) High-Volume, High-Velocity Data

Use edge processing, data compression and sampling, partitioned topics, windowed aggregations, scalable ingestion and tiered storage (hot, warm, cold).

(d) Scalability, Reliability and Security

- **Scalability and reliability:** Auto-scaling, multi-AZ brokers, message replication, retries and dead-letter queues.
- **Security:** Device certificates, TLS, secure boot, over-the-air updates, least-privilege policies and anomaly detection.

Set 3 - Question 3: BI Dashboards for E-Commerce

(a) BI Architecture

Sources (orders, web clicks, CRM) flow through streaming and batch pipelines into a cloud data warehouse with a semantic layer; BI tools (Power BI, Tableau, Looker, QuickSight) present dashboards.

(b) Visualisation and Warehouse Integration

BI tools connect through native connectors (JDBC/ODBC, live or import mode) to Redshift, BigQuery or Snowflake, use a shared semantic model for consistent metrics, and push queries down to the warehouse. Real-time views use streaming tables.

(c) Dashboard Performance

- **Techniques:** Pre-aggregated and materialised tables, extracts and caching, partitioning and clustering, incremental refresh, limiting visuals and filters, and query optimisation.

(d) Security Measures

- **Controls:** SSO and MFA, row-level and column-level security, role-based sharing, encryption in transit and at rest, data masking, audit logs and DLP.

Set 3 - Question 4: Gaming Monitoring and Optimisation

(a) Monitoring Solution

Centralised monitoring (CloudWatch, Prometheus / Grafana, Datadog) with metrics, logs, traces, real-user monitoring and alerts across game servers, databases and network.

(b) Finding Bottlenecks

- **Metrics:** CPU, memory, latency, packet loss, player count and error rates show where saturation occurs.
- **Logs:** Reveal errors and failed requests.
- **Tracing:** Distributed traces show which service adds delay.

(c) Resource Optimisation and Capacity Planning

Analyse historical peaks, apply predictive and scheduled auto-scaling, right-size instances, load test, and forecast growth.

(d) Availability and Cost Control

- **Methods:** Multi-region game servers placed near players, load balancing, health checks, CDN for assets, spot instances for non-critical workloads, reserved capacity for baseline, and budget alerts.

Set 3 - Question 5: Financial Data Lifecycle Management

(a) Lifecycle Strategy

Define stages: create and ingest, active use (hot), infrequent access (warm), archive (cold), and secure deletion. Automate transitions with lifecycle policies.

(b) Classification, Retention and Archival

- **Classification:** Label data by sensitivity and regulation (PII, financial, public) using automated discovery tools.
- **Retention:** Set periods by regulation (e.g., 7 years), with legal hold.
- **Archival:** Move to Glacier / Archive tiers with encryption and WORM (object lock); dispose securely when the period ends.

(c) Governance, Auditing and Compliance

Governance defines ownership, policies and accountability. Audit logs prove who accessed data; compliance with regulations (SOX, GDPR, PCI DSS) avoids fines and protects trust. Cataloguing and lineage make data traceable.

(d) Balancing Accessibility, Security and Cost

- **Approach:** Tier storage by access pattern, use intelligent tiering, compress, encrypt everything with managed keys, apply least-privilege access, and use query-in-place on archives to avoid costly restores.

Conclusion

The solutions above apply core cloud principles (elasticity, caching, orchestration, redundancy, defence in depth and automation) to each scenario. Each design decision is justified by performance, security, availability and cost requirements, which is what the rubric expects.